



Docker Cheatsheet

Based on the **specpart** E-Commerce Project — Beginner to Intermediate Guide

📦 13 Services · 2 Networks · 8 Volumes · CI/CD Pipeline

NestJS

Next.js

PostgreSQL

Redis

Kafka

OpenSearch

MinIO

nginx

GitHub Actions

01

🗨️ Core Concepts — The Mental Model

Think of Docker like **shipping containers** for software. Each concept maps to something real:



Image

A recipe / blueprint

specpart-backend:latest
postgres:16-alpine



Container

A running instance of an image

specpart-nginx
specpart-db



Volume

A USB drive — data persists

specpart_pgdata
kafka_data



Network

A private LAN between containers

frontend-net
backend-net



Dockerfile

Step-by-step build instructions

backend/Dockerfile
frontend/Dockerfile



docker-compose.yml

Orchestrates ALL services together

13 services in one file
start with one command

📌 KEY RULE

Images are read-only blueprints. **Containers** are live running instances — destroyed and recreated on every deploy. **Volumes** survive restarts, keeping your database data safe forever.

Dockerfile

dockerfile

```
# 1. Pull a base image from Docker Hub
FROM node:20-alpine

# 2. Set working directory inside container
WORKDIR /app

# 3. Copy package.json FIRST (cache trick!)
COPY package*.json ./

# 4. Install dependencies
RUN npm ci --only=production

# 5. Copy the rest of the source
COPY . .

# 6. Build the app
RUN npm run build

# 7. Document the port
EXPOSE 4000

# 8. Default startup command
CMD ["node", "dist/main.js"]
```

 CACHE TIP

Copy `package.json` and run `npm install` *before* copying source code. Docker caches layers — dependencies are only reinstalled when `package.json` changes. Huge speed boost!

INSTRUCTION	WHAT IT DOES
FROM	Base image to start from
WORKDIR	Working directory inside image
COPY	Copy files from host → image
RUN	Execute command during build
ENV	Set runtime env variables
ARG	Build-time variable (not at runtime)
EXPOSE	Document which port is used
CMD	Default command on startup
ENTRYPOINT	Like CMD but harder to override

Build-time ARGs in this project:

```
# From docker-compose.yml frontend service
build:
  context: .
  dockerfile: frontend/Dockerfile
  args:
    - NEXT_PUBLIC_API_URL=/api
    - NEXT_PUBLIC_SITE_URL=${NEXTAUTH_URL}
```

Instead of running 13 `docker run` commands, one file starts everything with `docker compose up -d`.

◆ Pre-built Image (Redis)

docker-compose.yml

yml

```
redis:
  image: redis:7-alpine      # From Docker Hub
  container_name: specpart-redis
  command: redis-server --appendonly yes
  volumes:
    - specpart_redis:/data   # Data persists!
  restart: unless-stopped
  networks:
    - backend-net           # Not public-facing
```

◆ Build from Dockerfile (Backend)

docker-compose.yml

yml

```
backend:
  image: specpart-backend:latest
  build:
    context: ./backend      # Where Dockerfile lives
    dockerfile: Dockerfile
  env_file: backend/.env    # Load secrets from file
  environment:
    - REDIS_HOST=redis      # Service name = hostname!
    - DATABASE_URL=postgresql://
      ${POSTGRES_USER}:${POSTGRES_PASSWORD}
      @db:5432/${POSTGRES_DB}
  depends_on:
    migrate:
      condition: service_completed_successfully
  redis:
    condition: service_healthy
```

 INTERNAL DNS MAGIC

`REDIS_HOST=redis` and `@db:5432` work because Docker's internal DNS resolves service names as hostnames. Containers on the same network find each other by name — no IP addresses needed!

◆ Healthchecks

```
backend:
  healthcheck:
    test: ["CMD", "wget", "--spider",
           "http://localhost:4000/api/health"]
    interval: 30s      # Check every 30s
    timeout: 10s       # Fail after 10s
    retries: 5         # Unhealthy after 5 fails
    start_period: 90s  # Grace period on startup
```

◆ One-Shot Container (Migrations)

```
migrate:
  image: specpart-backend:latest
  command: ["npx", "prisma",
            "migrate", "deploy"]
  depends_on:
    db:
      condition: service_healthy
  restart: "no" # Run ONCE, then exit
```

Networks in this project

```
networks:
  frontend-net:  # nginx + frontend
    driver: bridge
  backend-net:   # everything internal
    driver: bridge

# nginx bridges both networks
nginx:
  networks:
    - frontend-net
    - backend-net

# Redis is ONLY on backend-net (safe!)
redis:
  networks:
    - backend-net
```

⚠ SECURITY NOTE

Redis port 6379 is NOT mapped to the host. The compose file even has a comment: *"Do NOT expose port 6379 externally in production."*

Volumes in this project

VOLUME	STORES
specpart_pgdata	PostgreSQL primary data
specpart_pgdata_replica	PostgreSQL replica data
specpart_redis	Redis persistence
kafka_data	Kafka message log
opensearch_data	Search engine index
minio_data	Object storage files
certbot-etc/var	SSL certificates

VOLUME TYPE	SYNTAX	USE CASE
Named	myvolume:/path	Databases, state
Bind Mount	./host:/path	Config, dev hot-reload
Read-only	./file:/path:ro	Config (nginx uses this)

.env file

```
POSTGRES_USER=myuser
POSTGRES_PASSWORD=supersecret
POSTGRES_DB=specpart
JWT_SECRET=verylongrandomstring
NEXTAUTH_SECRET=anothersecret
```

Referencing in docker-compose.yml

```
environment:
  # Simple reference:
  - POSTGRES_USER=${POSTGRES_USER}

  # Required — fails loudly if missing:
  - POSTGRES_DB=${POSTGRES_DB:?DB is required}

  # With default fallback:
  - ADMIN_PW=${ADMIN_PW:-defaultpassword}

# Load all vars from a file:
env_file: backend/.env
```



● Image Commands

```
# Build from Dockerfile
docker build -t specpart-backend:latest ./backend

# List local images
docker images

# Remove an image
docker rmi specpart-backend:latest

# Pull from Docker Hub
docker pull postgres:16-alpine

# Save to file (used in CI/CD!)
docker save specpart-frontend:latest \
  specpart-backend:latest | gzip > images.tar.gz

# Load from file on remote server
docker load -i images.tar.gz

# Remove all unused images
docker image prune -a -f
```

● Network Commands

```
# List all networks
docker network ls

# Inspect (see connected containers)
docker network inspect backend-net

# Create manually (used in deploy.yml)
docker network create main_nexusaid-network || true
```

● Volume Commands

```
# List all volumes
docker volume ls

# Inspect a volume
docker volume inspect specpart_pgdata

# Remove a volume ⚠️
docker volume rm specpart_pgdata
```

● Container Commands

```
# List running containers
docker ps

# List ALL (including stopped)
docker ps -a

# Stop / Start / Restart
docker stop specpart-nginx
docker start specpart-nginx
docker restart specpart-backend

# Remove container
docker rm specpart-nginx
docker rm -f specpart-nginx # force remove running

# View logs
docker logs specpart-backend
docker logs -f specpart-backend # live tail
docker logs --tail 100 specpart-backend

# Open a shell inside a container
docker exec -it specpart-db bash
docker exec -it specpart-db sh # alpine

# Run one-off command
docker exec specpart-db pg_isready -U myuser
```

● Docker Compose Commands

```
# Start all services (background)
docker compose up -d

# Rebuild then start
docker compose up -d --build

# Start specific services only
docker compose up -d db redis

# Stop everything
docker compose down

# Stop + delete all volumes ⚠️
docker compose down -v

# View all logs live
docker compose logs -f

# Restart one service
docker compose restart backend

# Pull external images
docker compose pull db redis nginx

# Status of containers
docker compose ps

# With project name (used in this project)
docker compose -p achref up -d --no-build
```

07 Restart Policies

POLICY	BEHAVIOR	USED BY
no	Never restart (exits permanently)	migrate, minio-setup — run once and done
unless-stopped	Always restart EXCEPT if manually stopped. Survives reboots.	nginx, backend, frontend, db, redis ...
always	Restart always, even if manually stopped	Critical system services
on-failure	Only restart if container exits with error code	Batch jobs that should retry on error

From `.github/workflows/deploy.yml` — triggers on every `git push` to `main`:

deploy.yml — GitHub Actions

shell

```
# Step 1: Build images on GitHub runner
docker build -t specpart-backend:latest ./backend
docker build -t specpart-frontend:latest .

# Step 2: Save ALL images to one archive
docker save specpart-frontend:latest \
  specpart-backend:latest \
  specpart-migrate:latest | gzip > images.tar.gz

# Step 3: Clean old images on server (via SSH)
docker image prune -a -f --filter "until=24h"

# Step 4: Copy archive to server (via SCP)
scp images.tar.gz ubuntu@84.8.254.244:/opt/kiosquetn/

# Step 5: Load images on server (via SSH)
docker load -i images.tar.gz
rm -f images.tar.gz

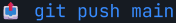
# Step 6: Pull lightweight external images
docker compose -p achref pull db redis nginx

# Step 7: Run DB migrations (one-shot)
docker compose -p achref up migrate

# Step 8: Bring up all services (no rebuild!)
docker compose -p achref up -d --no-build

# Step 9: Cleanup dangling images
docker image prune -f
```

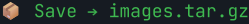
Deploy Pipeline



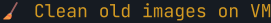
↓



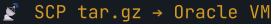
↓



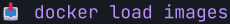
↓



↓



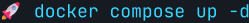
↓



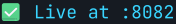
↓



↓



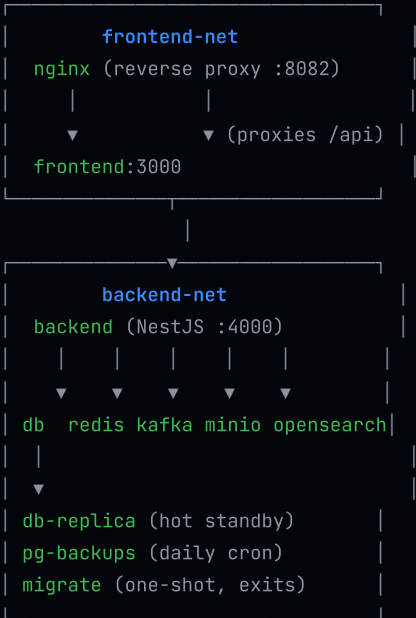
↓



 **--NO-BUILD**

This flag tells Compose to use the already-loaded images instead of building them again on the server. This is what makes deploys fast and consistent.

Internet → Port 8082 →



SERVICE	IMAGE	PURPOSE
nginx	nginx:alpine	Reverse proxy
certbot	certbot/certbot	SSL auto-renewal
frontend	specpart-frontend	Next.js app
backend	specpart-backend	NestJS REST API
migrate	specpart-backend	Prisma migrations
db	postgres:16-alpine	Primary database
db-replica	postgres:16-alpine	Hot standby reads
pg-backups	postgres-backup-local	Daily DB backups
redis	redis:7-alpine	Cache & sessions
minio	minio/minio	S3 object storage
minio-setup	minio/mc	Bucket creation
opensearch	opensearch:2.13.0	Full-text search
kafka	apache/kafka:3.7.0	Event messaging
kafka-ui	kafka-ui:latest	Kafka web UI

10

 Debugging Cheatsheet

Debugging Commands

```
# Why won't my service start?
docker compose logs backend

# Is the healthcheck passing?
docker inspect specpart-backend | grep -A 10 '"Health"'

# Open a shell to explore
docker exec -it specpart-backend sh

# Check env vars inside container
docker exec specpart-backend env

# Check port mappings
docker compose ps

# Live CPU / RAM usage
docker stats

# Inspect volume location
docker volume inspect specpart_pgdata

# Connect to PostgreSQL directly
docker exec -it specpart-db psql -U myuser -d specpart

# Flush all Redis keys
docker exec -it specpart-redis redis-cli FLUSHALL
```

• Cleanup Commands

```
# Remove stopped containers
docker container prune -f

# Remove unused images
docker image prune -a -f

# Remove unused volumes ⚠️
docker volume prune -f

# Remove unused networks
docker network prune -f

# Nuclear option - removes EVERYTHING ⚠️⚠️
docker system prune -a --volumes -f
```

⚠️ CAUTION

`docker system prune --volumes` deletes ALL data in unnamed/unused volumes. Named volumes like `specpart_pgdata` are safer but always double-check before pruning in production!

11

 Quick Reference Card

 Commands at a Glance

<code>docker build -t name .</code>	Build image from Dockerfile	<code>docker compose up -d</code>	Start all services (background)
<code>docker run -d -p 8080:80 img</code>	Run a single container	<code>docker compose up -d --build</code>	Rebuild then start all
<code>docker exec -it name sh</code>	Open shell inside container	<code>docker compose down</code>	Stop all services
<code>docker logs -f name</code>	Follow live container logs	<code>docker compose logs -f</code>	Follow all services logs
<code>docker ps</code>	List running containers	<code>docker compose ps</code>	Status of compose services
<code>docker images</code>	List all local images	<code>docker stats</code>	Live CPU / RAM usage
<code>docker save img gzip > f.tar.gz</code>	Export image to file	<code>docker compose restart svc</code>	Restart one service
<code>docker load -i file.tar.gz</code>	Import image from file	<code>docker system prune -a -f</code>	Remove all unused resources ⚠️